Computer Programming Assignment 2024

Veronica Capone 3240593

Abstract

The 3-SAT problem belongs to the complexity class of NP-Complete, therefore there is still no known algorithm that can solve it in polynomial time. This motivates the search for heuristic methods such as Simulated Annealing, which can search for a solution by using randomness.

The following study aims to evaluate the performance of Simulated Annealing in solving 3-SAT instances and to analyze the impact of different parameter settings and problem configurations on the estimation of the algorithmic thresholds.

Definition of the Problem

K-SAT is defined as a Constraint Satisfaction Problem (CSP) and it involves assigning values to a set of variables such that all predefined constraints are satisfied. Focusing on 3-SAT, we are given N binary variables $x = \{x_1, \ldots, x_N\}$ and M logical constraints, called clauses, each involving $K = 3$ variables from x, possibly negated. The domain of the binary variables is $\{0, 1\}^N$, where 0 represents *true* and 1 represents *false*.

A visual example of the following 3-SAT problem with $N = 4$ and $M = 3$:

$$\mathrm{SAT}_{3,3}(x) = (\neg x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_3 \vee x_1) \wedge (\neg x_4 \vee x_1 \vee \neg x_3),$$

where $\wedge$ indicates *and*, $\vee$ indicates *or*, and $\neg$ indicates *not*.

Optimization Setting

To find satisfying assignments for the 3-SAT problem, we implement Simulated Annealing, which is a heuristic optimization algorithm.
Unlike greedy algorithms, which always move toward improving solutions at each step, Simulated Annealing incorporates randomness through an acceptance function. This randomness allows the exploration of choices that may lead to higher costs under the assumption that they guide the search towards global optima rather than local minima.

Consequently, to solve K-SAT by using Simulated Annealing, it is necessary to introduce a change of variable:

$$\hat{x}_i = 1 - 2x_i \quad \text{for all } i \in \{1, \ldots, N\},$$

so that $\hat{x} \in \{-1, 1\}^N$, where 1 represents *true* and -1 represents *false*. Additionally, an appropriate cost function must be defined.

Cost Function

The cost function for the optimization problem is represented by the number of unsatisfied clauses:

$$E(\hat{x}) = \sum_{m=1}^{M} \prod_{k=1}^{K} \frac{1 - s_{mk} \cdot x_{mk}}{2},$$

where $s_{mk} \in \{-1, 1\}$ indicates the sign of the variables in each clause, and x_{mk} represents the assignment for that variable x that has been generated.

The cost function $E(x)$ is designed to evaluate whether a clause is satisfied or not, producing values of either 0 or 1. For a clause to be satisfied, it is sufficient that at least one of the variables within it matches the assignment generated.
In such cases, the corresponding s and x will have the same value (both -1 or both 1), resulting in:

$$\frac{1 - s_{mk} \cdot x_{mk}}{2} = 0.$$

This ensures that the contribution of the satisfied clause to the cost function is zero.

On the other hand, if none of the variables in the clause match the assignment, s and x will have opposite signs, leading to:

$$\frac{1 - s_{mk} \cdot x_{mk}}{2} = 1.$$

The product within the formula guarantees that the cost added for an unsatisfied clause is exactly 1.

In summary, $E(x)$ represents the total number of unsatisfied clauses, effectively quantifying the extent to which the assignment fails to satisfy the problem instance.

Implementation Details

The `cost()` function leverages vectorized operations such as `np.sum` and `np.prod` to ensure scalability for large problem sizes. In addition to the `self` parameter, the function takes `clauses` as an argument, allowing it to focus on specific clauses. This selective computation

is particularly useful when the `compute_delta_cost()` function is called.

The `compute_delta_cost()` function is designed to calculate the change in cost resulting from a proposed move. It achieves this by evaluating the cost before and after the move is accepted, ensuring that only the affected clauses are considered. This approach optimizes the overall computation process.

Parameter Choice

An appropriate selection of parameters such as β_0, β_1, the number of annealing steps, and mcmc_steps ensures a balance between exploring the solution space, avoiding entrapment in local minima, and exploiting promising solutions to refine the result. This balance is crucial for achieving both efficiency and accuracy, particularly for large-scale problems, as these parameters significantly impact runtime and solution quality.

The parameter $\boldsymbol{\beta_0}$, representing the initial inverse temperature of the annealing process, is determined to ensure an initial acceptance probability of cost-increasing moves of approximately 90%. A low $\boldsymbol{\beta_0}$ corresponds to a high temperature, enabling the algorithm to broadly explore the solution space by accepting even worse solutions (essential to escape local minima). For this purpose, an initial value of $\boldsymbol{\beta_0} = 0.1$ provides the desired level of acceptance rate at the start of the annealing process.

The parameter $\boldsymbol{\beta_1}$, representing the final inverse temperature at the end of the annealing process, focuses on refining the solution. A high β_1, which corresponds to low temperature, ensures that the algorithm is less likely to accept worse moves, improving solution accuracy and converging to a minimum. To achieve this, the annealing process should become more greedy over time, accepting moves that worsen the cost with decreasing probability.

With higher mcmc steps, there is an increased probability of accepting a move that worsens the current cost. However, when the algorithm reaches the critical inverse temperature β_c, the expected number of accepted cost-increasing moves per annealing step becomes less than one. To determine β_c, we use the formula for the probability of accepting cost-increasing moves:

$$P(\beta, \Delta c) = e^{-\beta \Delta c}$$

In the worst-case scenario, where all proposed moves increase the cost by 1, the expected number of accepted cost-increasing moves per annealing step is at most:

$$\text{Expected moves} = \text{mcmc_steps} \cdot e^{-\beta_c}$$

By enforcing the condition that this expectation is less than 1, we derive the inequality:

$$\beta_c > \ln(\text{mcmc_steps})$$

In our case, since mcmc_steps $= 8 \cdot N$, this simplifies to:

$$\beta_c > \ln(8 \cdot N)$$

In conclusion, we set $B1 = \ln(8 \cdot N) + 1$ to guarantee that the system has time to converge towards the minimum.

The number of **mcmc_steps** defines the number of iterations at each temperature. A higher number of mcmc_steps increases the likelihood of finding a better solution at each temperature. To account for the increased complexity of larger problem instances, the number of mcmc_steps is scaled with N as $8 \times N$, ensuring sufficient iterations as the number of variables increases.

The number of **annealing steps** determines how gradually the temperature decreases from β_0 to β_1. A higher number of steps gives the algorithm more time to explore the solution space thoroughly at each stage, making the process smoother and improving the likelihood of success. We fixed the number of annealing steps to 125. A higher number of annealing steps would have allowed for greater exploration of the solution space but would also have resulted in significantly higher computational costs.

The choice of using 125 annealing steps and $8 \times N$ mcmc_steps leads to a total of $125 \times 8 \times N = 1000 \times N$ proposed moves. This number is sufficient to achieve accurate results while balancing efficiency.
Additionally, the inclusion of a stopping condition ensures that the algorithm uses the annealing steps efficiently.
For instance, when M is low for a certain N, and a solution is found within a relatively small number of steps, the algorithm terminates early. Conversely, for more complex problems or when a solution does not exist, the algorithm utilizes all available steps.

Acceptance Rate

Behaviour of the acceptance rate with N = 200 and M = 200

To investigate the evolution of the acceptance rate during the optimization process, we considered the case where $N = 200$ and $M = 200$. The following parameter settings were used:

- $\beta_0 = 0.1$
- $\beta_1 = \ln(\text{mcmc_steps}) + 1$
- mcmc_steps $= 8 \times N = 8 \times 200 = 1600$
- annealing_steps $= 125$
- seed $= 42$ (by convention)

For $M = 200$, it was observed that the acceptance rate stabilizes around 0.6, both with and without the early stopping condition. This behavior can be attributed to the relatively low number of clauses, which results in most proposed moves having a delta cost equal to zero, as shown in the plot of Δc for $M = 200$. Consequently, the majority of moves are accepted by default.

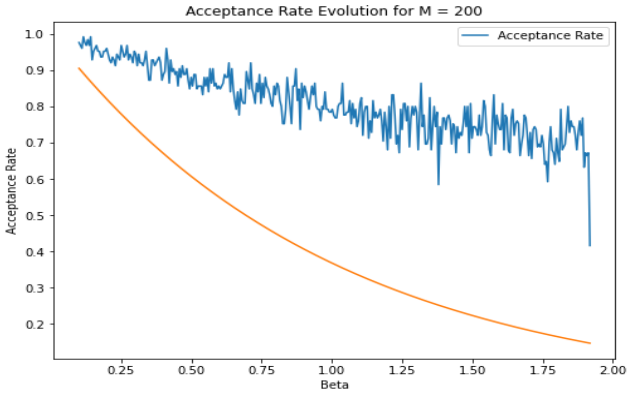


Figure 1: Acceptance rate with N = 200, M = 200

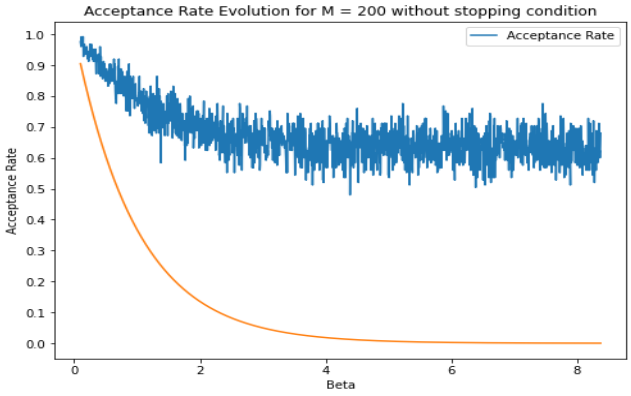


Figure 2: Acceptance rate with N = 200, M = 200 without early stopping condition

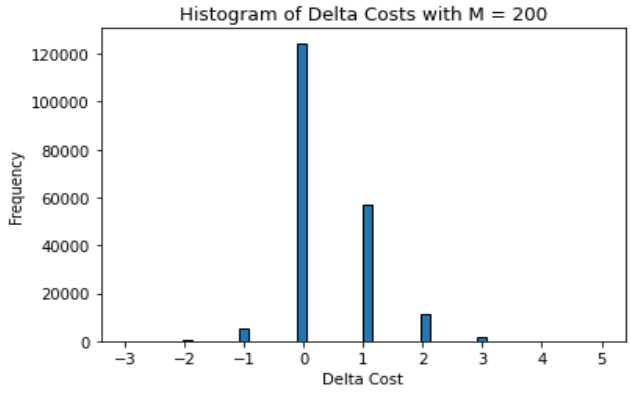


Figure 3: Delta cost with N = 200, M = 200

.

Behavior of the acceptance rate with M = [400, 600, 800, 1000]

Experiments with larger values of M reveal that Simulated Annealing does not always find a solution to the problem. For example, with $M = 1000$ and seed = 42, the algorithm consistently fails to find a solution. Even when changing the seed, the solution remains elusive. Regarding the acceptance rate, it tends to stabilize at lower values compared to $M = 200$, and it decreases more rapidly as M increases.

By analyzing the plot of Δc for different values of M, we observe that as M increases, the average Δc of proposed moves also increases. Consequently, the global acceptance rate heavily depends on the percentage of moves with $\Delta c > 0$ that are accepted. In the plots, in fact, you can find an orange line representing a function describing the probability of accepting a cost increasing move as beta increases. As M increases, the evolution of the acceptance rate progressively aligns more closely with the orange line.
Since the number of proposed moves with $\Delta c > 0$ is higher for larger M, the total number of accepted moves decreases, leading to a lower overall acceptance rate.

Moreover, increasing the number of clauses (M) reduces the number of global minima, thereby lowering the probability of finding one and solving the problem. Importantly, the inability to find a solution does not necessarily imply that a solution does not exist.

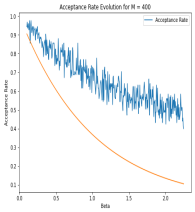


Figure 4: Acc.rate M=400

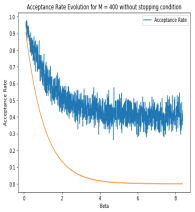


Figure 5: Acc.rate M=400 without stop. condition

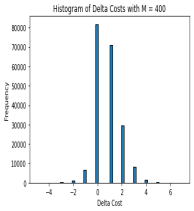


Figure 6: Delta cost M=400

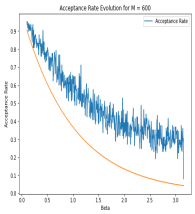


Figure 7: Acc.rate M=600

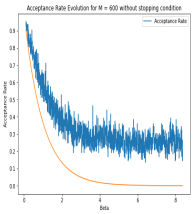


Figure 8: Acc.rate M=600 without stop. condition

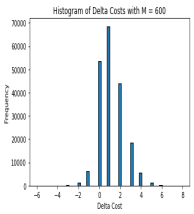


Figure 9: Delta cost M=600

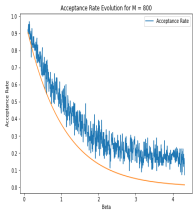


Figure 10: Acc.rate M=800

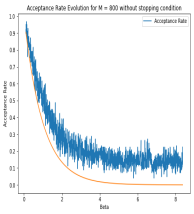


Figure 11: Acc.rate M=800 without stop. condition

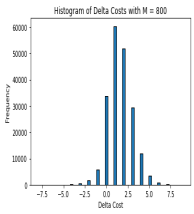


Figure 12: Delta cost M=800

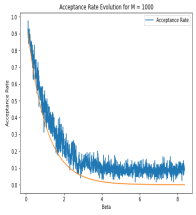


Figure 13: Acc.rate M=1000

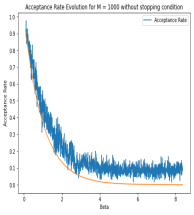


Figure 14: Acc.rate M=1000 without stop. condition

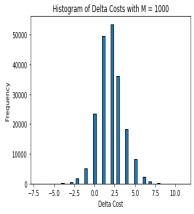


Figure 15: Delta cost M=1000

Algorithmic Threshold

Analysis of Algorithmic Threshold with N = 200

For $N = 200$, the commented code included in the file '3SAT properties' was used to identify the approximate algorithmic threshold. Observations indicate that the threshold lies between $M = 800$ and $M = 850$. Specifically, the probability $P(N, M)$ of solving the problem for $M = 800$ is 0.56 and drops below 0.5 for $M = 850$. Consequently, the probability $P(N, M) = 0.5$ is within the interval $800 < M < 850$.

To investigate the behavior of the algorithmic threshold for values of N other than 200, the same probability function was employed for $N = 300$. The results revealed that for $M = 850$ and $M = 900$, all problem instances were successfully solved. This suggests that the algorithmic threshold depends on N and increases with N.

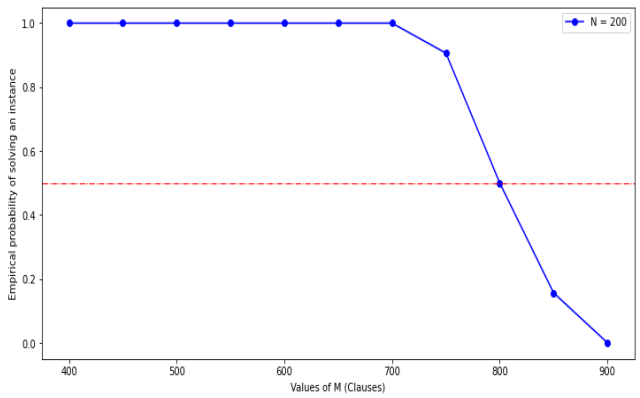


Figure 16: Algorithmic Threshold for N = 200

Relationship Between N and the Algorithmic Threshold

To further explore the relationship between N and the algorithmic threshold, modifications were implemented in the "probability" function to simultaneously analyze the threshold behavior for different values of N.

To accelerate the process, multiprocessing was utilized to perform operations in parallel. This required an auxiliary function "single_iteration" to handle individual instances.

The purpose of the probability function is to identify the values of M for which the algorithm fails to find a solution for a given N, thereby locating the algorithmic threshold. The process is repeated 5 times for 30 problem instances at each value of M, reducing randomness dependence. Once the 5 repetitions are complete, the process is repeated with M incremented by 50. If two failures are observed ($P(N, M) = 0$ for two repetitions), the algorithm proceeds to the next N, starting from the last value of M, again incremented by 50.

The final result is a list of lists of lists, where each sublist contains the 5 computed probabilities for each M, grouped by N.

To generate the final plots, we used the median of the 5 probabilities for each M. (The median of means is a statistical technique used to improve the accuracy and robustness of an estimate).

The graphs show a clear, approximately linear relationship between the algorithmic threshold and N. Specifically, the coefficient of proportionality between the algorithmic threshold and N varies between 3.8 and 4.1.

According to the studies, this coefficient is expected to be around 4.26. The discrepancy, and the lack of precise linearity, probably arises from the constraints on computational resources, requiring the use of a limited number of annealing steps and mcmc_steps to manage the complexity of the time simulation.
Moreover, randomness also plays a role, even though we tried to put in place a series of measures to reduce its impact.
Considering the above-mentioned constraints, we are assuming linearity even though there seems to be a slight decline, which is a strong assumption but is confirmed, at least to some extent, by the data.

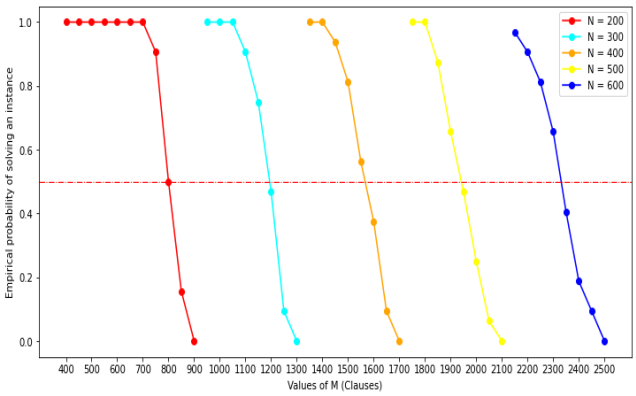


Figure 17: Algorithmic Threshold for N = [200, 400, 600, 800, 1000]

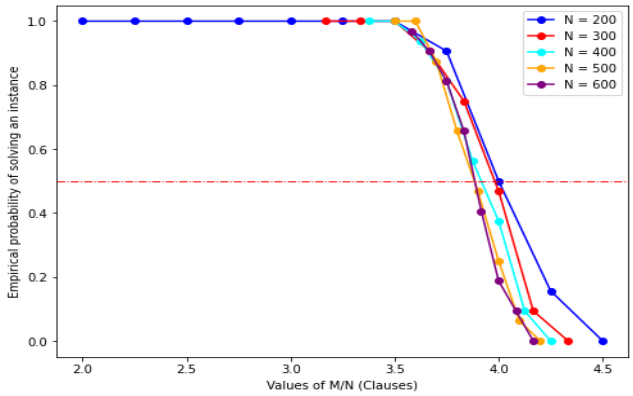


Figure 18: Collapse of the curves of P(N,M)

In figure 17 we can observe the behaviour of the probability function around the algorithmic threshold. In particular, it falls steeply from 1 to 0 as if it was passing through a phase transition. Moreover, the algorithmic threshold could be thought as the point where the existance of a solution for the system of variables and clauses starts to become uncertain.

In Figure 18, it can be observed that all the curves representing the probabilities P(N, M) collapse around a certain value of M/N, which is approximately 3.9. To understand better the relationship between N and the algorithmic threshold, we can calculate each coefficient empirically. First of all, thanks to the function `np.interp`, we can find the "most precise" M for which the probability is exactly equal to 0.5.
In our case, we have obtained the following list:

[800, 1194.444, 1566.667, 1941.667, 2331.25]

where the position of each M found is given by N (in order $N = [200, 300, 400, 500, 600]$).

Then, we calculated the coefficient M_{Alg}/N for each setting as follows:

- For $N = 200$, $M_{\text{Alg}}/N = \frac{800}{200} = 4$
- For $N = 300$, $M_{\text{Alg}}/N = \frac{1194.444}{300} \approx 3.981$
- For $N = 400$, $M_{\text{Alg}}/N = \frac{1566.667}{400} \approx 3.917$
- For $N = 500$, $M_{\text{Alg}}/N = \frac{1941.667}{500} \approx 3.883$
- For $N = 600$, $M_{\text{Alg}}/N = \frac{2331.25}{600} \approx 3.885$

The ratio remains approximately constant at ≈ 3.9, suggesting that the algorithmic threshold scales linearly with N and that the general coefficient is around 3.9. This relationship provides valuable information on how the complexity of the problem evolves as N increases. Specifically, as the number of variables N grows with constant M, the average number of clauses in which each variable participates decreases. This reduction in clause density enhances the system's flexibility, as individual variables are less constrained by overlapping requirements. Consequently, finding configurations that satisfy all constraints becomes easier.

A Enlarged reproduction of figures used within the text

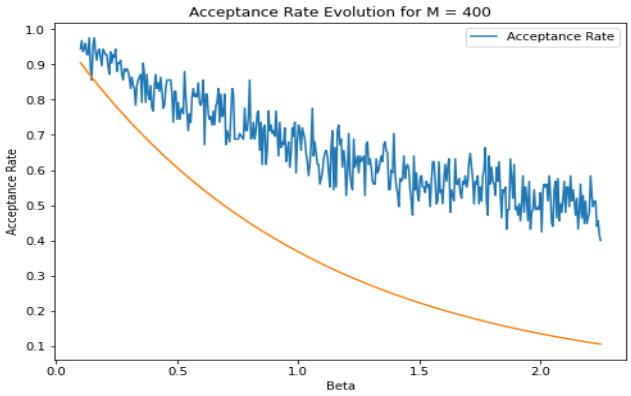


Figure 5: Acceptance rate for M=400

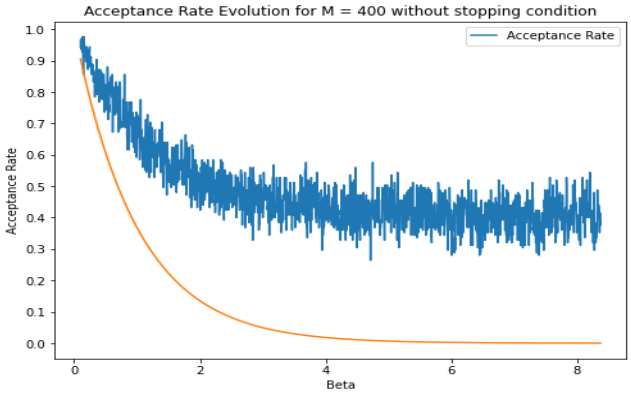


Figure 6: Acceptance rate for M=400 without early stopping condition

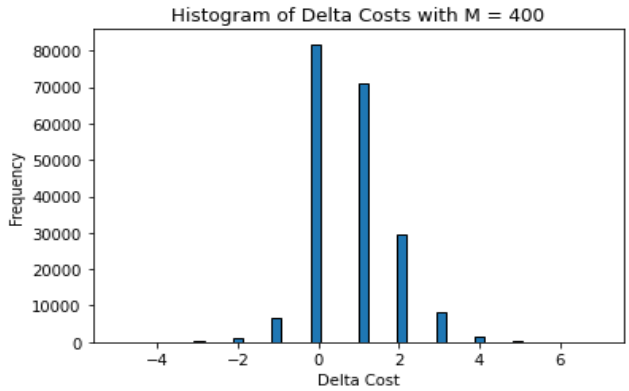


Figure 7: Delta cost for M=400

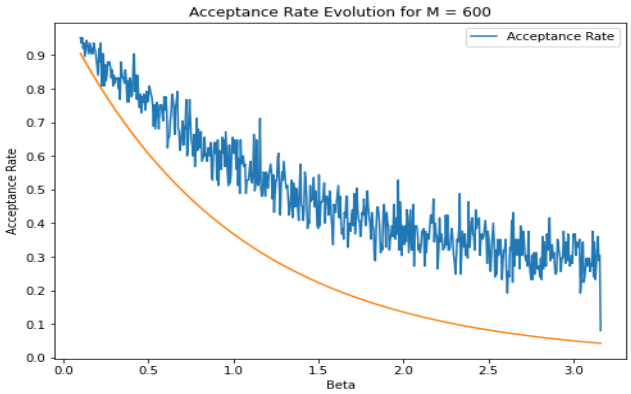


Figure 8: Acceptance rate for M=600

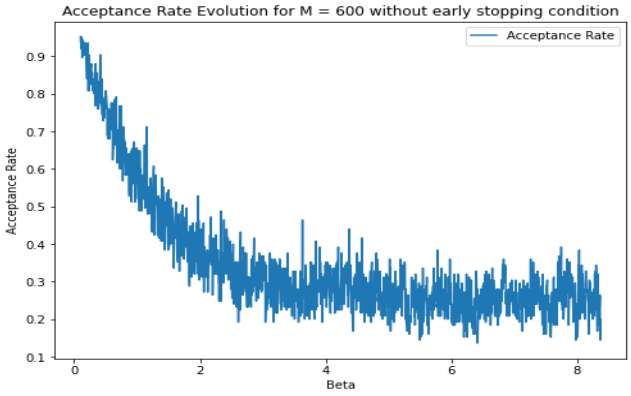


Figure 9: Acceptance rate for M=600 without early stopping condition

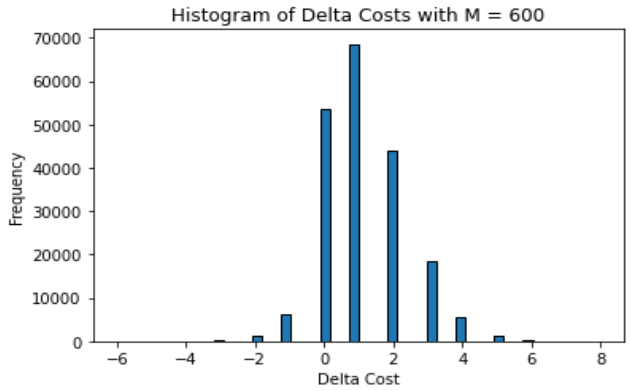


Figure 10: Delta cost for M=600

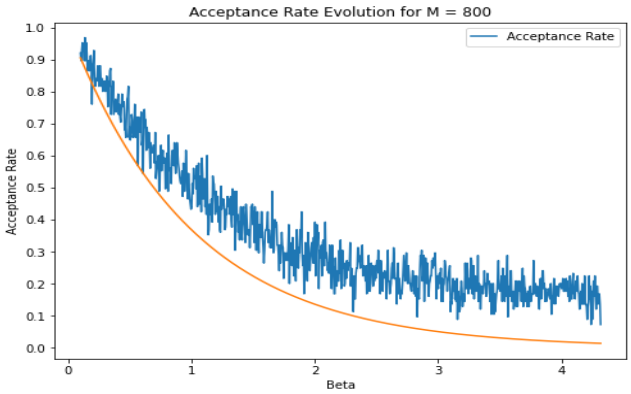


Figure 11: Acceptance rate for M=800

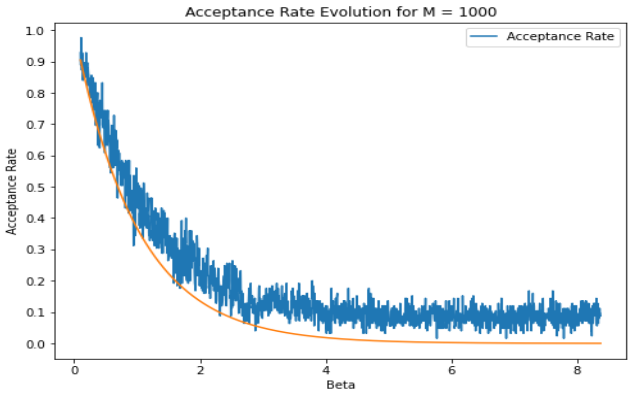


Figure 14: Acceptance rate for M=1000

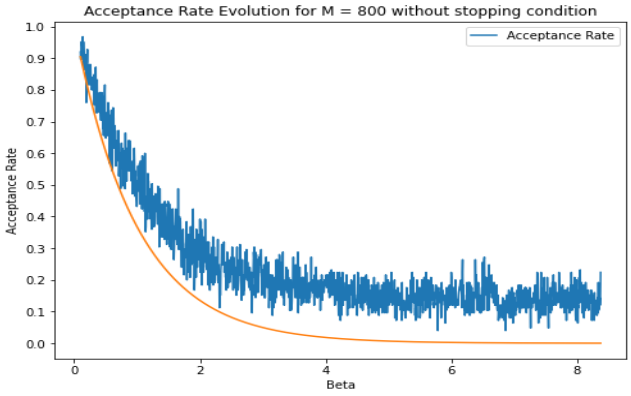


Figure 12: Acceptance rate for M=800 without early stopping condition

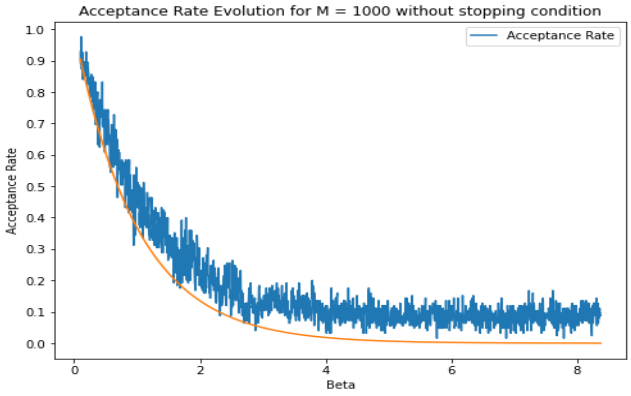


Figure 15: Acceptance rate for M=1000 without early stopping condition

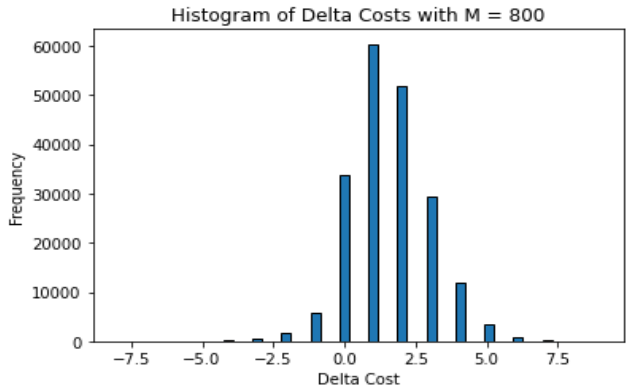


Figure 13: Delta cost for M=800

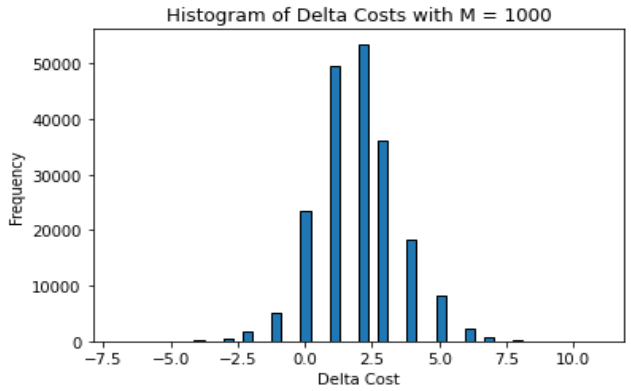


Figure 16: Delta cost for M=1000